Zagazig University
Faculty of Engineering
Electronics and Communications Engineering Department

AES Report

Information Security 2025

Prepared by:

1- Hisham Mohamed Abdelrahman
2- Ali Mohamed Fawzy
3- Adel Mohamed Ahmed
4- Abdallah Mohamed Ahmed
5- ziad Ahmed Abdelrahman
6- ziad Ayman Abdulsalam
7- Omar Talaat Mohamed Ahmed
8- Mohamed Ashraf Mohamed Embaby
9- Ziad Ashmawy El-Demerdash

Introduction:

In 1997, the National Institute of Standards and Technology (NIST) issued a public request for proposals to develop a new symmetric encryption standard to replace the aging Data Encryption Standard (DES). In 1998, fifteen candidate algorithms were submitted, although five were shown to have vulnerabilities. In 1999, NIST selected five finalists, and in 2000 the Rijndael algorithm----designed in Belgium Was chosen to become the AES standard.

AES supports key sizes of 128, 192 and 256 bits, while the block size is fixed at 128-bit.

AES as a Substitution -Permutation (SP) Network:

AES is not a Feistel cipher. Instead, it is based on **Substitution-Permutation Network (SPN).**

An SP-Network operates by alternating two main layers:

a. **Substitution Layer:**

Each byte passes through S-boxes (S1…S8), providing nonlinearity and confusion

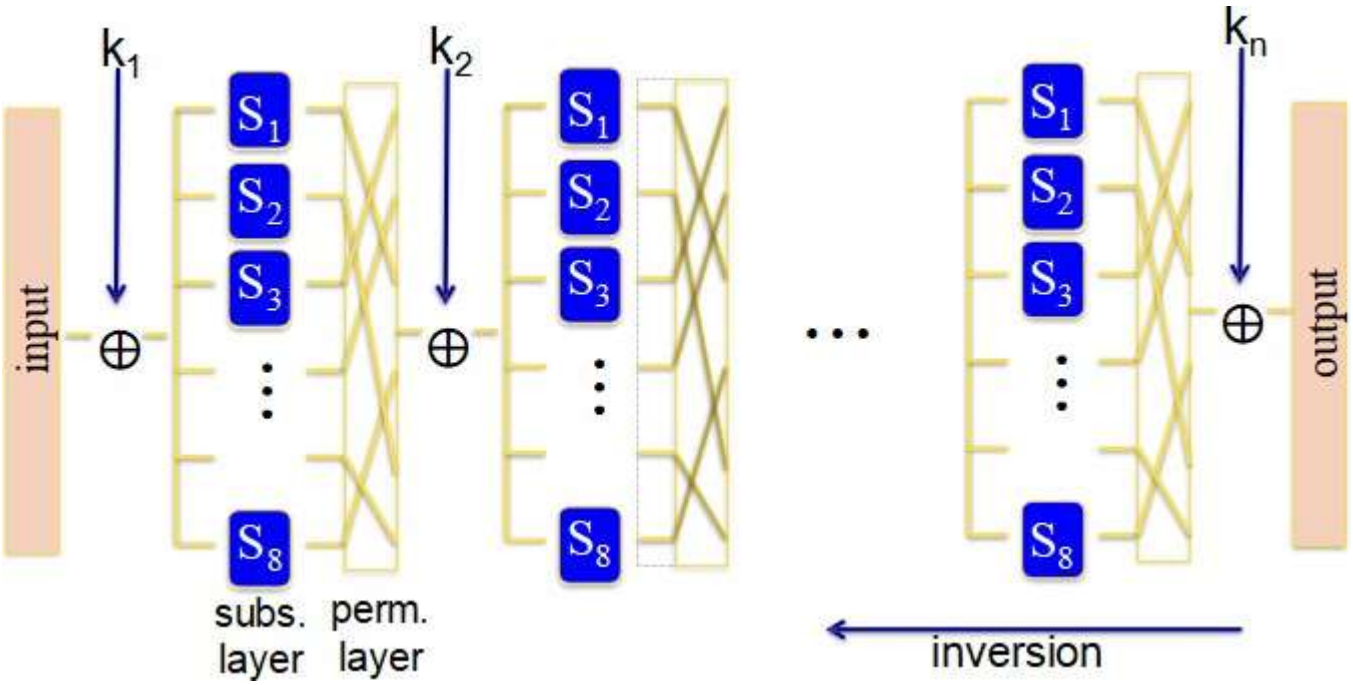
b. **Permutation Layer:**

A deterministic rearrangement of bytes or rows introduces diffusion across the data block.

c. **Key Mixing**

Each round combines the internal state with a round key using XOR.

The overall structure is:



This forms the basis of AES round transformations.

AES-128 Round Structure:

AES-128 operates on a 4×4 byte matrix over 10 rounds.

a. Round Operations

Each round consists of:

1. ByteSub: Byte-wise substitution using a S-Box.
2. ShiftRows: Cyclic row shifts.
3. Mixcolumns: Column-level mixing using arithmetic in GF(2^8).
4. AddRoundKey: XOR with the round key.

b. Initial and final Rounds

1. Round 0: Only AddRoundKey (k_0).
2. Rounds 1-9: ByteSub →ShiftRows →MixColumns→ AddRoundKey.
3. Round 10: ByteSub →ShiftRows →AddRoundKey.

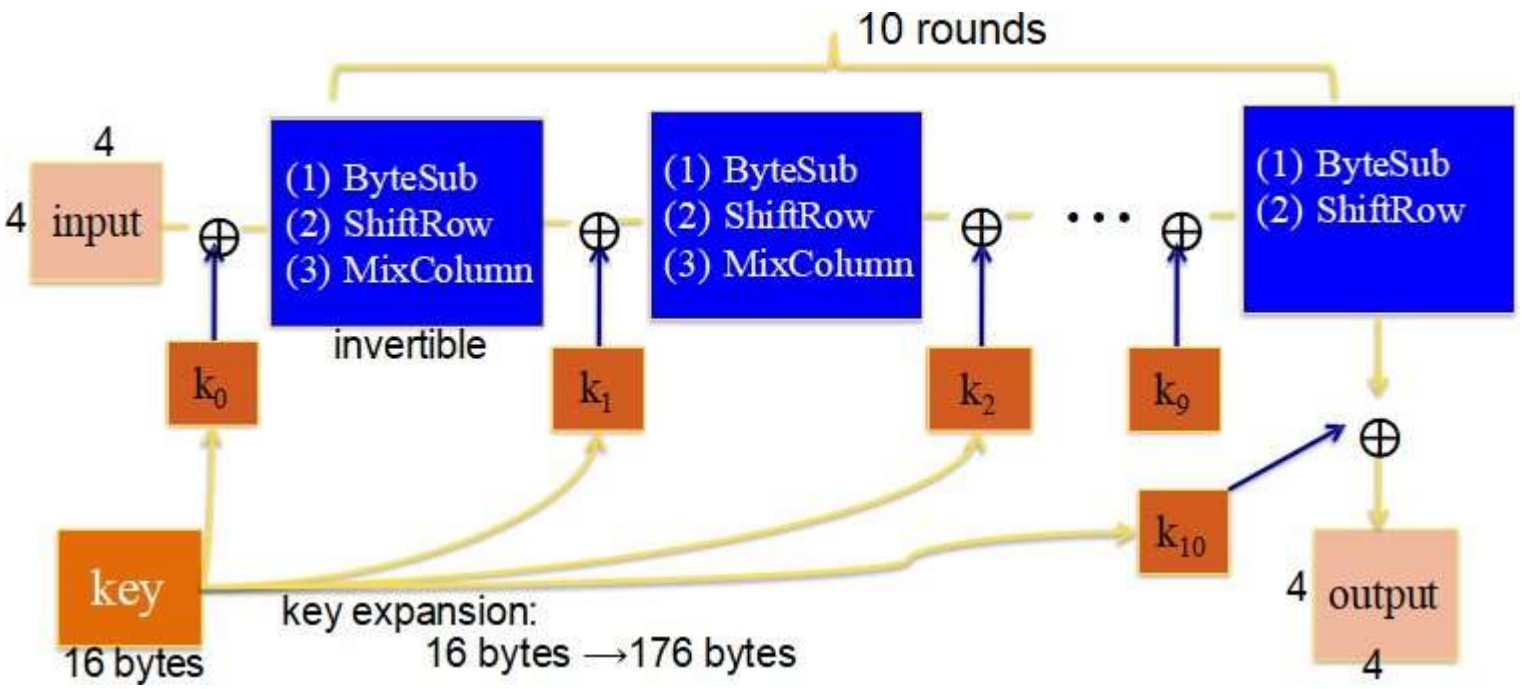
.

c. Key Expansion

The 16-byte AES-128 key is expanded into **176 bytes**, forming **44 words (32-bit each).**

Every set of 4 words (128-bit) is used as a round key.

The overall structure is:



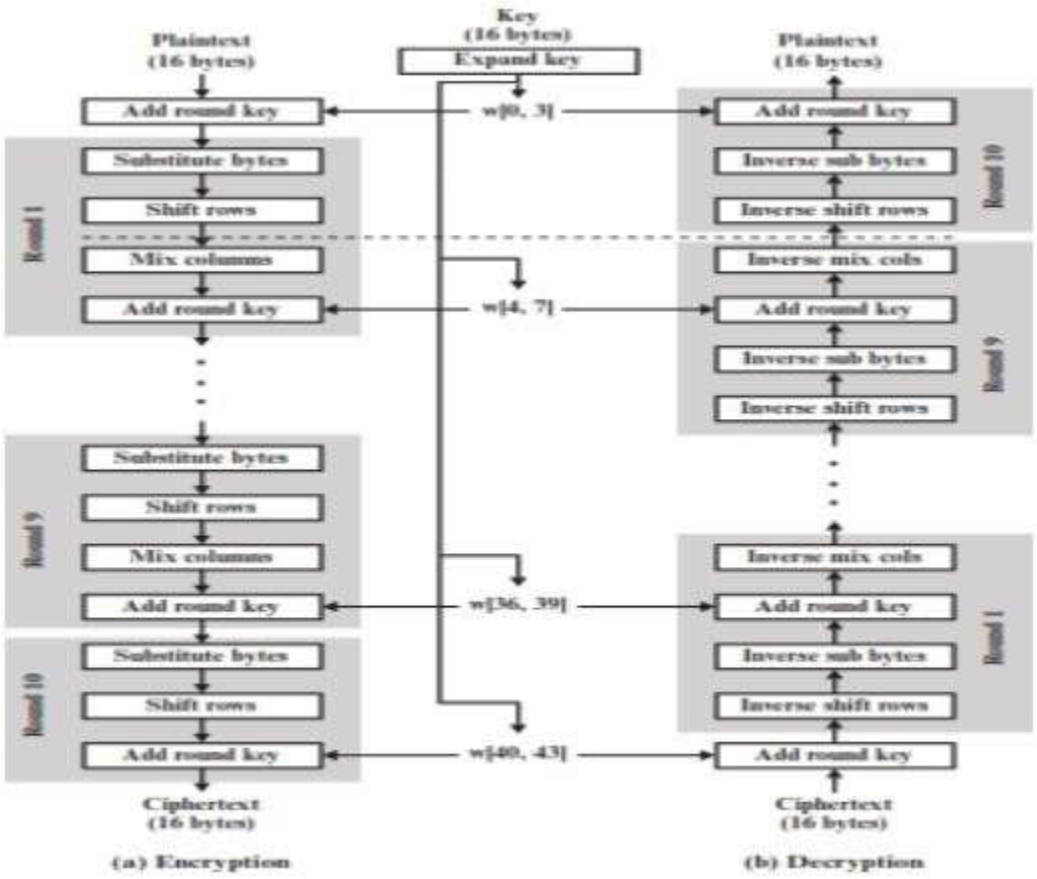
Overall AES Structure:

AES processes the entire 128-bit block as a single matrix per round. The expanded key is structured into 44 words w[i]. Four consecutive words from the round key.

The four transformations used in AES serve distinct cryptographic purposes:

- **SubstituteBytes** →Nonlinear substitution
- **ShiftRows** →Permutation
- **MixColumns** → Diffusion through linear mixing
- **AddRoundKey** →Key-dependent transformation.

Encryption beings with AddRoundKey, followed by 9 full rounds and a final reduced round.



Reversibility and Decryption Mechanism :

Every AES stage is reversible

- **Inverse ByteSub:** Uses inverse S-box.
- **Inverse ShiftRows:** Reverse row rotations.
- **Inverse MixColumns:** Multiplication by inverse matrix in GF.
- **AddRoundKey:** is its own inverse due to XOR.

$$A \oplus B \oplus B = A$$

Decryption applies the expanded key in reverse order, beginning with the last round key.

Implementation Tradeoffs:

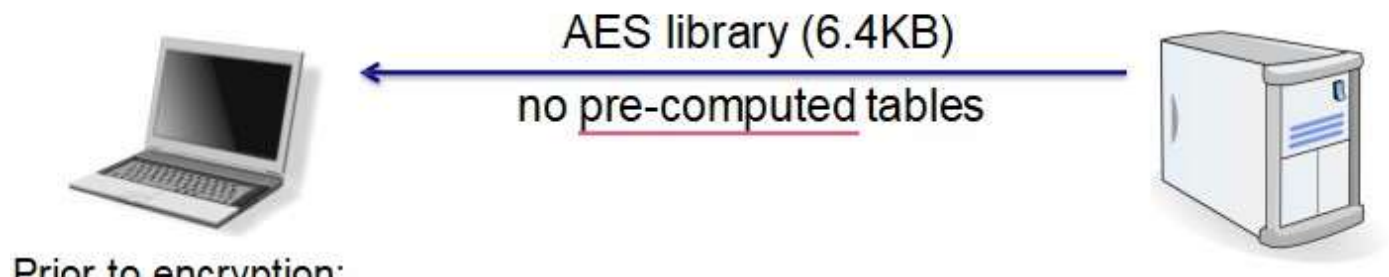
a. Code Size vs. Performance

AES implementation may vary based on lookup tables:

Implementation	Code size	Performance
Pre-compute round functions (24KB or 4KB)	largest	fastest: table lookups and xors
Pre-compute S-box only (256 bytes)	smaller	slower
No pre-computation	smallest	slowest

b. Javascript AES Example:

AES in the browser:



Prior to encryption:
pre-compute tables

Then encrypt using tables

http://crypto.stanford.edu/sjcl/

AES in Hardware:

Modern processors (Intel Westmere, AMD Bulldozer) implement AES instructions:

- **Aaesenc, aesenclast**--perform one AES round on 128-bit registers.
- **Aeskeygenassist**--hardware key expansion.
- These provide up to **14× speed up** over software implementations such as OpenSSL.

Known Attacks:

AES remains secure in practice, but several theoretical attacks exist:

- Best key recovery attack: ~4× faster than exhaustive search (BKR'11).
- Related-key attack on AES-256 (BK'09): Using 2^{99} input/output pairs under related keys, the key can be recovered in ~2^{99} time.

These attacks do **not** threaten real-world AES usage.

Block Ciphers and PRGs:

A block cipher maps plaintext blocks to ciphertext blocks in a manner resembling randomness.
Thus, block ciphers are suitable candidates for building pseudorandom generators (PRGs).

Constructing PRFs from PRGs:

- For any block of plaintext, a symmetric block cipher produces an output block that is apparently random.
- That is, there are no patterns or regularities in the ciphertext that provide information that can be used to deduce the plaintext.
- Thus, a symmetric block cipher is a good candidate for building a pseudorandom number generator

AES Implementation in Python:

This section presents a simple educational implementation of AES-128 encryption and decryption using Python. The code uses the **`PyCryptodome`** library, which provides a reliable and efficient implementation of the AES block cipher. The programs operate in Electronic Codebook (ECB) mode for demonstration purposes.

a. AES Encryption Code

The following Python program encrypts a plaintext message using a 16-byte AES key.

```
from Crypto.Cipher import AES
import sys
def pad(text):
    pad_len = 16 - (len(text) % 16)
    return text + chr(pad_len) * pad_len
text = sys.argv[1]        # text
key = sys.argv[2]         # key
if len(key) != 16:
      print("Error: Key must be exactly 16 characters long.")
      sys.exit(1)
text_padded = pad(text)
cipher = AES.new(key.encode(), AES.MODE_ECB)
ciphertext = cipher.encrypt(text_padded.encode())
print(ciphertext.hex())
```

b. AES Decryption Code

The following program decrypts the ciphertext generated by the encryption script. The ciphertext is given as a hexadecimal string.

```
from Crypto.Cipher import AES
import sys
def unpad(text_bytes):
    pad_len = text_bytes[-1]
    return text_bytes[:-pad_len]
if len(sys.argv) < 3:
    print("Error: Missing arguments. Usage: python aes_decrypt.py <cipher_hex>
<key>")
    sys.exit(1)
cipher_hex = sys.argv[1]   # Ciphertext in hex
key = sys.argv[2]           # key
if len(key) != 16:
    print("Error: Key must be exactly 16 characters long.")
    sys.exit(1)
try:
    ciphertext = bytes.fromhex(cipher_hex)
    cipher = AES.new(key.encode(), AES.MODE_ECB)
    plaintext_padded = cipher.decrypt(ciphertext)
    plaintext = unpad(plaintext_padded).decode()
```

```
    print(plaintext)
except Exception as e:
    print(f"Error: {str(e)}")
```

c. Libraries Used

1. PyCryptodome (Crypto.Cipher.AES)
   Provides the AES cipher implementation, including multiple encryption modes. Used to create AES objects and perform encrypt/decrypt operations.

2. sys Module
   A built-in Python library used to handle command-line arguments, detect invalid inputs, and exit the program safely.

3. Standard Python Functions
   Used for padding, removing padding, handling exceptions, and converting hexadecimal strings to byte arrays.

d. Advantages of the Implementation

1. Simplicity and clarity
   The code is minimal and easy to understand, making it suitable for students learning encryption fundamentals.

2. Direct control of padding
   Manual PKCS#7-style padding helps illustrate how block ciphers handle short plaintexts.

3. Lightweight dependencies
   Only one external library (PyCryptodome) is required.

4. AES-128 compliance
   The code correctly enforces a 16-byte (128-bit) key length.

5. Predictable output
   Ciphertext is displayed in hexadecimal, making it easy to test and re-use.

e. Limitations and Challenges

1. Use of ECB Mode (Security Weakness)

ECB mode does not hide patterns in the plaintext, making it insecure for real-world use.
It is included here only for educational demonstration.

2. Lack of Authentication
   The implementation does not provide integrity or authenticity, unlike modes such as AES-GCM.

3. Manual Padding Risks
   Incorrect handling of padding can lead to vulnerabilities such as padding-oracle attacks.

4. Fixed Key Size (128-bit Only)
   AES-192 or AES-256 are not supported in this basic version.

5. No IV or Randomness
   Since ECB mode does not use an initialization vector, repeated plaintext leads to repeated ciphertext.

6. Not Suitable for Large Data
   The program processes input as a single string rather than in secure chunks, limiting use for big files.

Referances:

- **Cryptography and Network Security Principles and Practice(William Stallings).**

- **mastering python for network.**